

React Router Hands-On Lab — Solution

Building the Cognizant Academy Trainers Directory ("TrainersApp") with React Router

Objectives

- Explain the need and benefits of React Router
- Identify the components in React Router
- List the types of Router components
- Parameter passing via URL

In this hands-on lab, you will learn how to:

- Implement a simple navigation menu
- Add basic routes (install, configure)
- Use routes in React applications

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

The Cognizant Academy team wants to maintain a list of trainers along with their expertise in a Single-Page Application built with React. Each trainer record has a T-ID, Name, Phone, Email, Stream, and Skills. The application must display a Home page, a list of all trainers (clickable by name), and a details page for a selected trainer, all navigable through React Router.

Solution — Step by Step

Step 1: Create the React project "TrainersApp"

```
npx create-react-app TrainersApp
```

Step 2: Open the application in VS Code

```
cd TrainersApp  
code .
```

Step 3: Create the Trainer class in Trainer.js

Add a new file named Trainer.js inside the src folder that defines the Trainer class:

```
class Trainer {  
  constructor(trainerId, name, email, phone, technology, skills) {  
    this.trainerId = trainerId;  
    this.name = name;
```

```
    this.email = email;
    this.phone = phone;
    this.technology = technology;
    this.skills = skills;
  }
}

export default Trainer;
```

Step 4: Create the mock trainer data in TrainersMock.js

Add a new file named TrainersMock.js inside the src folder with a set of sample trainers:

```
import Trainer from './Trainer';

const TrainersMock = [
  new Trainer(
    1,
    'Ravi Kumar',
    'ravi.kumar@cognizant.com',
    '9876543210',
    '.NET',
    ['C#', 'ASP.NET Core', 'Azure']
  ),
  new Trainer(
    2,
    'Sneha Iyer',
    'sneha.iyer@cognizant.com',
    '9876543211',
    'Java',
    ['Core Java', 'Spring Boot', 'Microservices']
  ),
  new Trainer(
    3,
    'Arjun Mehta',
    'arjun.mehta@cognizant.com',
    '9876543212',
    'Frontend',
    ['React', 'JavaScript', 'CSS']
  ),
];

export default TrainersMock;
```

Step 5: Install React Router DOM

Run the following command to add routing support to the project:

```
npm install react-router-dom
```

Step 6: Create the TrainersList component

Add a new file named TrainersList.js. The component accepts the trainer data as a prop and renders each trainer's name as a clickable link:

```
import React from 'react';
import { Link } from 'react-router-dom';

function TrainersList({ trainers }) {
  return (
    <div>
      <h2>Trainers List</h2>
      <ul>
        {trainers.map(trainer => (
          <li key={trainer.trainerId}>
            <Link to={` /trainers/${trainer.trainerId}`}>
              {trainer.name}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TrainersList;
```

Step 7: Create the Home component

Add a new file named Home.js that displays a welcome message for the portal:

```
import React from 'react';

function Home() {
  return (
    <div>
      <h2>Welcome to the Cognizant Academy Trainers Portal</h2>
      <p>Use the navigation menu above to view the list of trainers.</p>
    </div>
  );
}

export default Home;
```

Step 8: Add routing and navigation links in App.js

Update App.js to import BrowserRouter, Routes, Route, and Link from react-router-dom, define the navigation menu, and set up the / and /trainers routes:

```
import React from 'react';
import {
```

```

    BrowserRouter,
    Routes,
    Route,
    Link
  } from 'react-router-dom';
import Home from './Home';
import TrainersList from './TrainersList';
import TrainersMock from './TrainersMock';
import './App.css';

function App() {
  return (
    <BrowserRouter>
      <div className="App">
        <nav>
          <Link to="/">Home</Link> | <Link to="/trainers">Trainers</Link>
        </nav>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route
            path="/trainers"
            element={<TrainersList trainers={TrainersMock} />}
          />
        </Routes>
      </div>
    </BrowserRouter>
  );
}

export default App;

```

Step 9: Create the TrainerDetail component

Add a new file named TrainerDetail.js. This component reads the id parameter from the URL using the useParams hook, looks up the matching trainer in the mock data, and displays that trainer's details:

```

import React from 'react';
import { useParams } from 'react-router-dom';
import TrainersMock from './TrainersMock';

function TrainerDetail() {
  const { id } = useParams();
  const trainer = TrainersMock.find(
    t => t.trainerId === parseInt(id, 10)
  );

  if (!trainer) {
    return <p>Trainer not found.</p>;
  }
}

```

```

    return (
      <div>
        <h2>Trainer Details</h2>
        <p><strong>T-ID:</strong> {trainer.trainerId}</p>
        <p><strong>Name:</strong> {trainer.name}</p>
        <p><strong>Email:</strong> {trainer.email}</p>
        <p><strong>Phone:</strong> {trainer.phone}</p>
        <p><strong>Stream:</strong> {trainer.technology}</p>
        <p><strong>Skills:</strong> {trainer.skills.join(', ')}</p>
      </div>
    );
  }

  export default TrainerDetail;

```

The Link created in TrainersList.js in Step 6 already points to /trainers/\${trainer.trainerId}, so no change is needed there. Now add a matching route for TrainerDetail in App.js:

```

import TrainerDetail from './TrainerDetail';

// inside <Routes> ...
<Route path="/trainers/:id" element={<TrainerDetail />} />

```

Complete App.js

Putting all the routes together, the final App.js looks like this:

```

import React from 'react';
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from 'react-router-dom';
import Home from './Home';
import TrainersList from './TrainersList';
import TrainerDetail from './TrainerDetail';
import TrainersMock from './TrainersMock';
import './App.css';

function App() {
  return (
    <BrowserRouter>
      <div className="App">
        <nav>
          <Link to="/">Home</Link> | <Link to="/trainers">Trainers</Link>
        </nav>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route

```

```
        path="/trainers"
        element={<TrainersList trainers={TrainersMock} />}
      />
      <Route path="/trainers/:id" element={<TrainerDetail />} />
    </Routes>
  </div>
</BrowserRouter>
);
}

export default App;
```

Step 10: Build and run the application

```
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

The Home link (/) displays the welcome message. Clicking Trainers (/trainers) shows the list of trainer names as clickable links. Clicking a trainer's name navigates to /trainers/:id and displays that trainer's T-ID, Name, Email, Phone, Stream, and Skills, confirming that navigation, route configuration, and URL parameter passing are all working correctly.

Result

A React application named "TrainersApp" was successfully created using create-react-app. The Trainer class models each trainer's data, and TrainersMock.js supplies sample records. react-router-dom was installed and used to configure three routes — Home (/), the trainers list (/trainers), and trainer details (/trainers/:id) — wired together with BrowserRouter, Routes, Route, and Link. The TrainerDetail component retrieves the selected trainer's id from the URL using the useParams hook and displays the corresponding trainer's full details. The application was verified by running it locally and navigating between all three views at http://localhost:3000.